Instituto Politécnico Formosa

Trabajo Práctico Integrador N° III

Objetivo:

Desarrollar desde cero una API REST para gestionar usuarios, eventos y categorías, usando MongoDB como base de datos NoSQL y Mongoose como ODM. El proyecto debe aplicar todas las tecnologías vistas en clase: JWT, cookies, bcrypt, validaciones con express-validator, relaciones embebidas y referenciadas, virtual populate y operaciones CRUD completas con eliminación lógica y en cascada.

Criterios de Evaluación

1. Presentación del código

- El código debe estar limpio, ordenado y bien indentado.
- Uso obligatorio de try-catch en todos los controladores para un manejo adecuado de errores.
- Organización correcta del proyecto en carpetas:
    - src/config → configuración (conexión a MongoDB).
    - src/models → definición de esquemas Mongoose.
    - src/routes → definición de rutas.
    - src/controllers → controladores.
    - src/middlewares → autenticación, autorización, validaciones y manejo de errores de validación.
    - src/helpers → utilidades (JWT, bcrypt).
- Uso exclusivo de ESModules (import / export).
- El código debe ser funcional, modularizado y sin errores de ejecución.

2. Validaciones y lógica

- Validaciones implementadas con express-validator y con las validaciones de esquema de Mongoose.
- Uso de validationResult() en un middleware reutilizable que devuelva los errores con código 400.
- Uso de matchedData() en los controladores para trabajar únicamente con los datos validados.
- Respuestas claras y códigos HTTP apropiados en cada operación:
    - 201 Created → al crear.
    - 200 OK → en consultas y actualizaciones exitosas.
    - 400 Bad Request → errores de validación.
    - 401 Unauthorized → falta autenticación.
    - 403 Forbidden → falta autorización.
    - 404 Not Found → recurso inexistente.
    - 500 Internal Server Error → errores inesperados.
- Verificación de unicidad al crear o editar recursos con campos únicos.
- Verificación de existencia previa antes de editar o eliminar un recurso.
- Mensajes de éxito o error al realizar operaciones CRUD.

3. Autenticación y Autorización

- Sistema JWT completo: generación, verificación y manejo de tokens.
- Cookies seguras con httpOnly.
- Hasheo de contraseñas con bcrypt en el registro y el login.
- Middleware de autenticación para proteger las rutas privadas.
- Middleware de autorización con permisos diferenciados para users y admins.
- Controladores de autenticación completos: registro, login, logout y perfil.

4. Relaciones, virtual populate y eliminación

- Al menos dos documentos embebidos (1:1).
- Al menos una relación referenciada 1:N.
- Al menos una relación N:M mediante un array de referencias.
- Uso de populate en las colecciones que guardan la referencia.
- Uso de virtual populate (ref, localField, foreignField) en las colecciones que no guardan la referencia.
- Configuración de **toJSON y toObject con { virtuals: true }** en los esquemas que tengan virtual populate.
- Eliminación lógica implementada en al menos un modelo.
- Eliminación en cascada implementada en al menos dos relaciones.

Entrega mediante Git y GitHub – Uso de ramas y commits

Requisitos obligatorios para el control de versiones

- Crear un nuevo repositorio llamado **“trabajo-practico-integrador-3”** con un README inicial en la rama **main**.
- Crear la rama **develop** desde **main**.
- Crear la rama **integrador-eventos** desde develop y realizar todo el desarrollo en ella.
- Durante el desarrollo en la rama **integrador-eventos**, realizar al menos **10 commits** con mensajes claros y descriptivos.
- Al finalizar el trabajo:
    - Hacer un merge limpio, sin conflictos, de **integrador-eventos** hacia **develop**.
    - Luego hacer un merge limpio de develop hacia **main**, para que ambas ramas queden sincronizadas.

Requisitos obligatorios del repositorio remoto

- El repositorio debe contener todos los archivos del proyecto (app.js, package.json, src/ completo).
- Debe incluir el archivo .env.example con los nombres de las variables utilizadas.
- El archivo .env y la carpeta node_modules no deben subirse.
- El enlace al repositorio debe entregarse en Google Classroom.

Consignas

1. Configuración inicial del proyecto

- Inicializar el proyecto Node.js con npm init.
- Instalar las dependencias necesarias:
  - express, mongoose, cors, dotenv
  - jsonwebtoken, bcrypt, cookie-parser, express-validator
- Crear la estructura de carpetas según los criterios de evaluación.
- Configurar las variables de entorno (.env): MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN y PORT.
- Crear el archivo de conexión a la base de datos con Mongoose.
- Configurar el servidor Express con los middlewares básicos (CORS, cookie-parser, parser de JSON).

2. Esquemas y relaciones

Crear los siguientes modelos. Todos deben usar timestamps: true y versionKey: false.

1. User (Usuario)

- username (String, único, 3-20 caracteres)
- email (String, único, formato válido)
- password (String, hasheada con bcrypt)
- role (String, enum: 'user', 'admin', default: 'user')
- **profile (documento embebido, relación 1:1):**
  - firstName (String, 2-50 caracteres)
  - lastName (String, 2-50 caracteres)
  - birthDate (Date, opcional)
  - phone (String, opcional)
- deletedAt (Date, default: null)
- **Virtual populate events → eventos organizados por el usuario.**

2. Event (Evento)

- title (String, único, 3-150 caracteres)
- description (String, mínimo 20 caracteres)
- startDate (Date, obligatorio)
- endDate (Date, obligatorio)
- capacity (Number, entero, mínimo 1)
- price (Number, mínimo 0, default: 0)
- status (String, enum: 'scheduled', 'cancelled', 'finished', default: 'scheduled')
- **location (documento embebido, relación 1:1):**
  - venue (String, 3-100 caracteres)
  - address (String, 5-150 caracteres)
  - city (String, 2-50 caracteres)
- **organizer (ObjectId, referencia a User, relación 1:N)**
- **categories (array de ObjectIds, referencias a Category, relación N:M)**

3. Category (Categoría)

- name (String, único, 2-30 caracteres)
- description (String, opcional, máximo 200 caracteres)
- **Virtual populate events → eventos que tienen la categoría en su array categories.**

Relaciones a implementar:

- 1:1 embebida: User ↔ Profile.
- 1:1 embebida: Event ↔ Location.
- 1:N referenciada: User → Event (organizer).
- N:M referenciada: Event ↔ Category (array de ObjectIds en Event).

Eliminación lógica

- User: usar el campo deletedAt para el soft delete.
- Las consultas normales deben ignorar a los usuarios con deletedAt distinto de null.

Eliminación en cascada

- User: al eliminar lógicamente un usuario, se deben eliminar todos los eventos que organizó.
- Category: al eliminar una categoría, se debe quitar de todos los eventos que la referencian.

3. Sistema de autenticación y autorización

Crear los helpers de utilidad:

- **JWT Helper:** funciones para generar y verificar tokens JWT.
- **Bcrypt Helper:** funciones para hashear y comparar contraseñas.

Implementar los middlewares:

- **authMiddleware:** verificar el JWT desde las cookies y extraer el usuario.
- **adminMiddleware:** verificar que el usuario autenticado sea admin.
- **ownerOrAdminMiddleware:** verificar que el usuario sea el organizador del evento o sea admin.

Desarrollar los controladores de autenticación:

- POST /api/auth/register → Registro de usuario con perfil embebido. ***(público)***
- POST /api/auth/login → Login con JWT enviado como cookie segura. ***(público)***
- GET /api/auth/profile → Perfil del usuario autenticado. ***(usuario autenticado)***
- PUT /api/auth/profile → Actualizar el perfil embebido del usuario autenticado. ***(usuario autenticado)***
- POST /api/auth/logout → Logout limpiando la cookie de autenticación. ***(usuario autenticado)***

4. Controladores con CRUD completo

Users:

- GET /api/users → Listar los usuarios activos con el virtual populate events. ***(solo admin)***
- GET /api/users/:id → Obtener un usuario activo con sus eventos. ***(solo admin)***
- PUT /api/users/:id → Actualizar un usuario. ***(solo admin)***
- DELETE /api/users/:id → Eliminación lógica del usuario con eliminación en cascada de sus eventos. ***(solo admin)***

Events:

- POST /api/events → Crear un evento con sus categorías; el organizador es el usuario autenticado. ***(usuario autenticado)***
- GET /api/events → Listar los eventos con populate de organizer y categories. ***(usuario autenticado)***
- GET /api/events/my → Listar los eventos del usuario logueado. ***(usuario autenticado)***
- GET /api/events/:id → Obtener un evento con populate de organizer y categories. ***(usuario autenticado)***
- PUT /api/events/:id → Actualizar un evento y sus categorías. ***(solo organizador o admin)***
- DELETE /api/events/:id → Eliminación física de un evento. ***(solo organizador o admin)***

Categories:

- POST /api/categories → Crear una categoría. ***(solo admin)***
- GET /api/categories → Listar las categorías. ***(usuario autenticado)***
- GET /api/categories/:id → Obtener una categoría con el virtual populate events. ***(usuario autenticado)***
- PUT /api/categories/:id → Actualizar una categoría. ***(solo admin)***
- DELETE /api/categories/:id → Eliminar una categoría con eliminación en cascada. ***(solo admin)***

5. Validaciones con express-validator y Mongoose

- Validar campos obligatorios, tipos de datos y longitudes en las rutas de creación y actualización.
- Validar los campos de los documentos embebidos **(profile y location).**
- Validar con isMongoId() los IDs recibidos por parámetro y en el cuerpo de la petición.

Validaciones específicas por modelo:

User/Auth:

- username: 3-20 caracteres, alfanumérico, único.
- email: formato válido, único.
- password: mínimo 8 caracteres, con al menos una mayúscula, una minúscula y un número.
- role: solo valores permitidos ('user', 'admin').

Profile (embebido):

- firstName y lastName: 2-50 caracteres, solo letras.
- birthDate: fecha válida, no futura (opcional).
- phone: solo números, entre 8 y 15 dígitos (opcional).

Event:

- title: 3-150 caracteres, obligatorio, único.
- description: mínimo 20 caracteres, obligatorio.
- startDate y endDate: fechas válidas, obligatorias.
- capacity: entero mayor o igual a 1.
- price: número mayor o igual a 0.
- status: solo valores permitidos ('scheduled', 'cancelled', 'finished').
- categories: **array de ObjectIds válidos (opcional).**

Location (embebido):

- venue, address y city: obligatorios, con las longitudes indicadas en el esquema.

Category:

- name: 2-30 caracteres, único, obligatorio.
- description: máximo 200 caracteres.

Validaciones personalizadas con custom():

- endDate debe ser posterior a startDate.
- Solo el organizador puede editar o eliminar sus eventos (excepto admin).
- Las categorías enviadas en categories deben existir.

Entrega

- Repositorio en GitHub con todas las ramas requeridas.
- Código funcional sin errores.
- Archivo .env.example con las variables requeridas.
- Al menos 10 commits descriptivos en la rama de desarrollo.